

Credit Card Default (Taiwan)

Machine Learning Project Report

Student ID	Name	Section No.
2300479	Mohamed Mohamed Abd Elhamid	3
2300397	Abdelrahman Yasser	2
2300824	Mahmoud Mohamed Mokhtar	3

April 24, 2026

Contents

1	Problem Framing and Data Understanding	2
1.1	Problem Being Solved	2
1.2	Target Variable	2
1.3	Classification Type	2
1.4	Expected Challenges	2
2	Data Acquisition & Documentation	3
2.1	Dataset Source	3
2.2	Modifications Made	3
3	Data Cleaning & Preprocessing	4
3.1	Handling Missing Values	4
3.2	Encoding Categorical Variables	4
3.3	Scaling Numerical Features (If Needed)	4
3.4	Handling Class Imbalance (If Needed)	4

1 Problem Framing and Data Understanding

1.1 Problem Being Solved

This project solves a supervised machine learning problem: predicting whether a credit card client will default on the next month's payment based on demographic information, repayment history, bill statements, and payment amounts.

1.2 Target Variable

The target variable is `default payment next month`. In simple terms, this means whether a person is expected to fail to make the required credit card payment on time in the next billing cycle.

To make this clear for readers with no finance background:

- A credit card has a *minimum payment* due each month.
- If a customer does not pay that required amount by the due date, this is considered a **default** for that period.
- The model predicts this event one month ahead.

Although the dataset was collected in Taiwan, the meaning of default is not country-specific. It represents a general credit-risk event used in banking worldwide.

Label encoding in the dataset is:

- 0 = non-default
- 1 = default

1.3 Classification Type

This is a **binary classification** task because the target has two classes only: default or non-default.

1.4 Expected Challenges

The main expected challenges in this dataset are:

- **Feature engineering needs:** important predictive signals are spread across monthly billing, payment, and repayment-status columns, so engineered aggregate and ratio features are needed.
- **Missing values handling:** missing or unclear entries can reduce model reliability and must be checked and treated carefully.
- **Feature selection decisions:** the dataset contains many related variables, so we must decide which original and engineered features to keep to reduce redundancy and improve generalization.
- **Class imbalance risk:** defaults are typically fewer than non-defaults, so evaluation should emphasize metrics like precision, recall, and F1-score, not accuracy alone.
- **Data noise and coding inconsistencies:** categorical fields may include uncommon/invalid codes and require controlled recoding before modeling.

2 Data Acquisition & Documentation

2.1 Dataset Source

The dataset used in this project is:

- **Default of Credit Card Clients (UCI Machine Learning Repository)**
- Link: <https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients>

In implementation, the dataset is fetched programmatically from UCI through `ucimlrepo` (dataset id 350), then converted into a working dataframe for preprocessing and modeling.

2.2 Modifications Made

The following modifications were applied to the raw data:

- **Column-name standardization:** original fields (e.g., `X1..X23`, `Y`, and alternative target naming) were mapped to consistent readable names such as `LIMIT_BAL`, `PAY_0`, and `default payment next month`.
- **Target normalization:** the target was normalized to a one-dimensional series and unified under one label: `default payment next month`.
- **Repayment-status harmonization:** `PAY_*` variables were aligned to a shared category space, then remapped into semantic categories (e.g., `paid_in_full`, `revolving`, `delay_2_months`, `delay_3_plus_months`).
- **Categorical recoding:** `EDUCATION`, `MARRIAGE`, and `SEX` were recoded into cleaner categorical labels.
- **Specific label merge:** `EDUCATION` values 5 and 6 were merged into 4 (others) before final category labeling.
- **Invalid category filtering:** rows with `EDUCATION = 0` or `MARRIAGE = 0` were removed as invalid category codes.
- **Duplicate handling policy:** duplicates were intentionally **kept** in the current notebook version (no deduplication removal step).
- **Feature engineering:** aggregate and behavioral features were added, including totals, trends, ratios, and utilization features (e.g., `TOTAL_BILL_AMT`, `AVG_UTILIZATION`, `MAX_UTILIZATION`).
- **Feature-selection cleanup:** redundant predictors were removed (e.g., `BILL_AMT2-BILL_AMT6`, `UTILIZATION_2-UTILIZATION_6`, and `ID` when present).
- **Train/test split:** data was split using stratified 80/20 train-test partition with fixed random seed 42 to preserve class balance.
- **Scaling transformation:** `StandardScaler` was fit on training numeric columns only, then applied to test data to avoid leakage.

3 Data Cleaning & Preprocessing

3.1 Handling Missing Values

Missing values were audited using a data-quality summary table that reported missing counts and percentages per column. In this workflow, no standalone imputation method (such as mean/median/mode filling) was applied. Instead, data quality control was handled by:

- validating target and schema consistency,
- filtering invalid categorical codes (e.g., `EDUCATION = 0`, `MARRIAGE = 0`), and
- preserving valid records for downstream feature engineering.

3.2 Encoding Categorical Variables

Categorical preprocessing focused on semantic recoding and category harmonization:

- repayment status columns (`PAY_0` to `PAY_6`) were aligned to a shared category space,
- repayment values were remapped into ordered semantic categories (e.g., `paid_in_full`, `revolving`, `delay_3_plus_months`),
- `EDUCATION`, `MARRIAGE`, and `SEX` were recoded to clean categorical labels,
- `EDUCATION` values 5 and 6 were merged into 4 (others) before final labeling.

At this stage, categorical variables were standardized as categorical features; a separate full one-hot encoding pipeline was specified for later modeling stages.

3.3 Scaling Numerical Features (If Needed)

Numerical scaling was applied using `StandardScaler` after train/test split:

- numeric columns were selected from `X_train`,
- scaler was fit only on training data,
- the same fitted scaler was used to transform both training and test sets.

This prevents information leakage from test data into training.

3.4 Handling Class Imbalance (If Needed)

Class distribution was checked during the data audit and EDA phases. The notebook handled imbalance conservatively by using a **stratified** train/test split so both classes keep similar proportions in train and test sets. No oversampling/undersampling method (e.g., SMOTE) was applied in this stage.